Paper Review • arXiv 2611.14488

Is Your Robot's Memory Just a Clock?

Temporal Shortcuts in Memory-Augmented VLA Policies

Chubin Zhang, Bo An • Nanyang Technological University • 2026-11

In one sentence Time-filled robot memory actually “reads the clock” to estimate progress: pauses, wrist fidgets and other “null events” that change nothing are counted as work, so counting tasks stop early. Removing timestamps or pause augmentation does not fix this; indexing memory by events recovers most of the loss without retraining.

Code: github.com/bennidict23/memory-clock

Can time-filled memory tell “waiting” from “working”?

Mainstream memory design: FrameSamp

- Uniformly sample 32 frames from the full trajectory
- Each frame gets a sinusoidal encoding of its absolute timestep
- Fixed cost covers the whole history; strongest released policy on RoboMME (built on π0.5)

But time passes even when nothing happens

Real robots wait for slow planners, operator pauses and safety-monitor clearance; they also hesitate and grasp at air.

Null event

Time passes and the robot may even move, yet nothing the task depends on changes. An ideal event memory should “ignore” it.

3 null event types, injected after policy steps 48 & 160

Pause

Hold in place 100 env steps (5 s); dose study: 30 / 300 steps

Fidget

Wrist rolls ±0.3 rad about the hold target for 100 steps

Air-grasp

Grasp air above a static object; return to pre-event pose

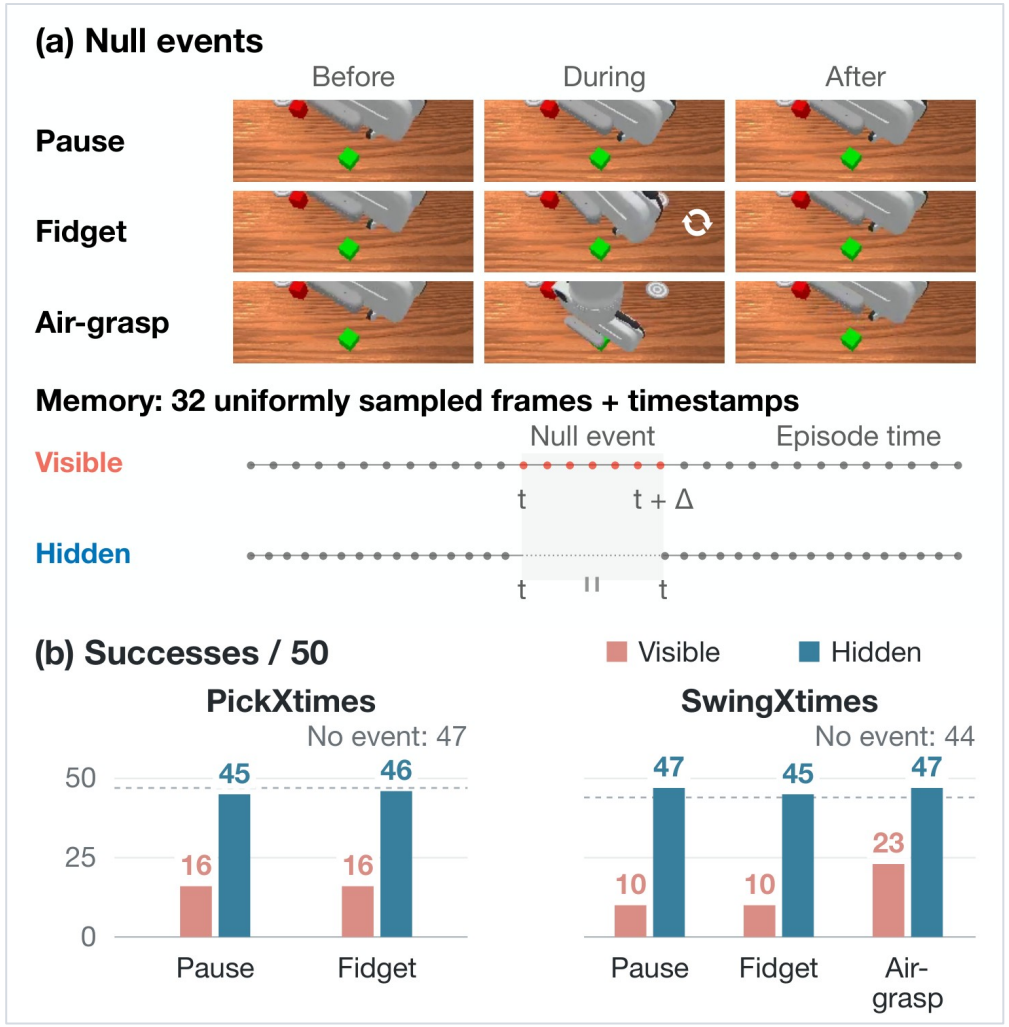
Key control: Visible vs Hidden

Visible: event frames enter memory; timestamps advance past it

Hidden: same physical event, but frames not stored, timestamps frozen

Phenomenon

Two 5 s pauses collapse success, harming only via memory



PickXtimes • Pause

47 → 16

Pause frames hidden: 45 / 50

SwingXtimes • Pause

44 → 10

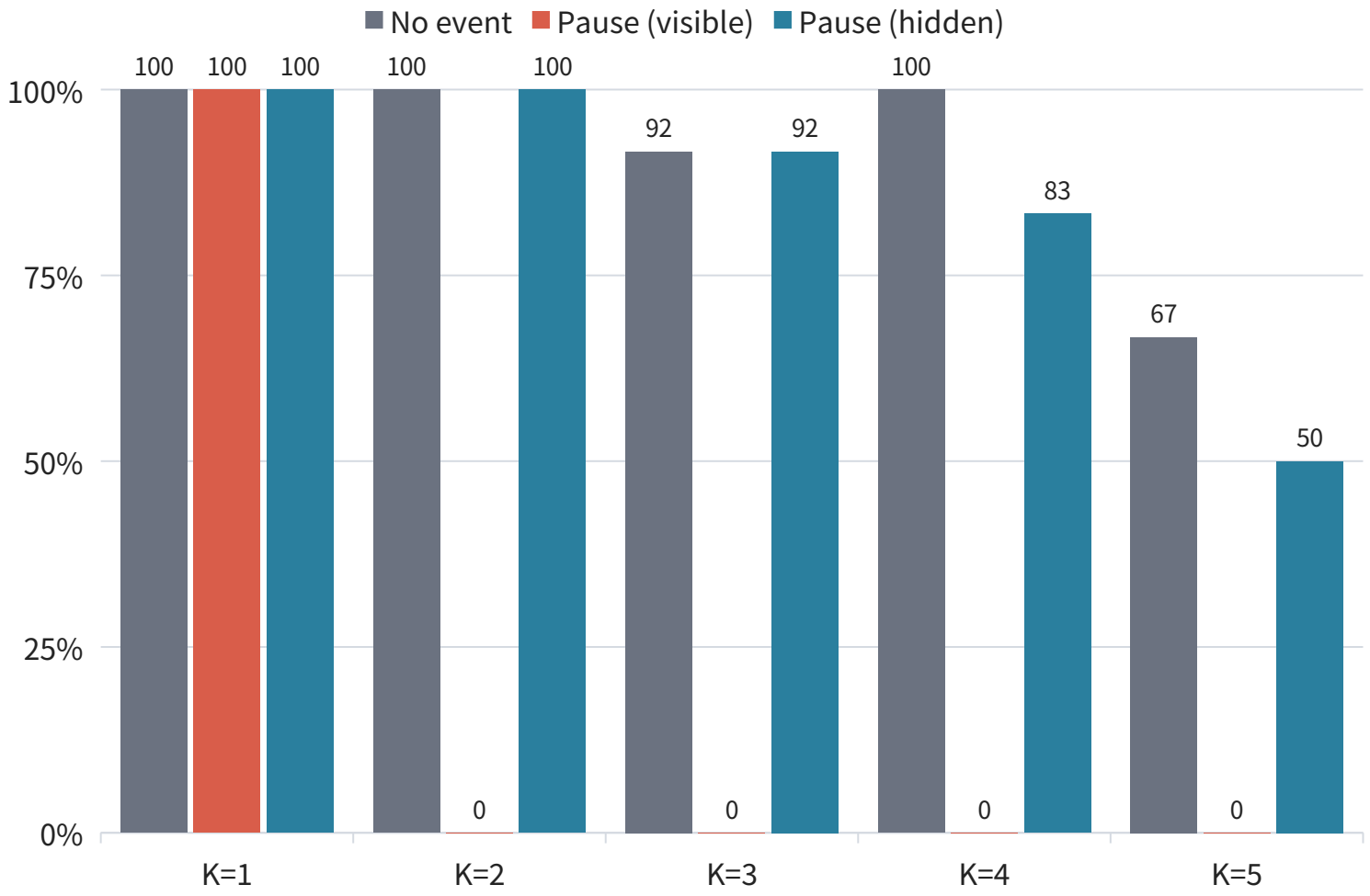
Pause frames hidden: 47 / 50

Phantom progress

Fidget is equally harmful (16 / 10); air-grasp drops SwingXtimes to 23. Nearly all failures are button presses before the K-th rep.

Phenomenon

Only “when to stop” episodes fail: every K⩾2 episode fails



Y-axis: success rate (%). Episodes: K=1..5 have 16 / 10 / 12 / 6 / 6

K=1 • Pause

16/16 → 16/16

First stop was timed correctly anyway

K⩾2 • Pause/Fidget

31/34 → 0/34

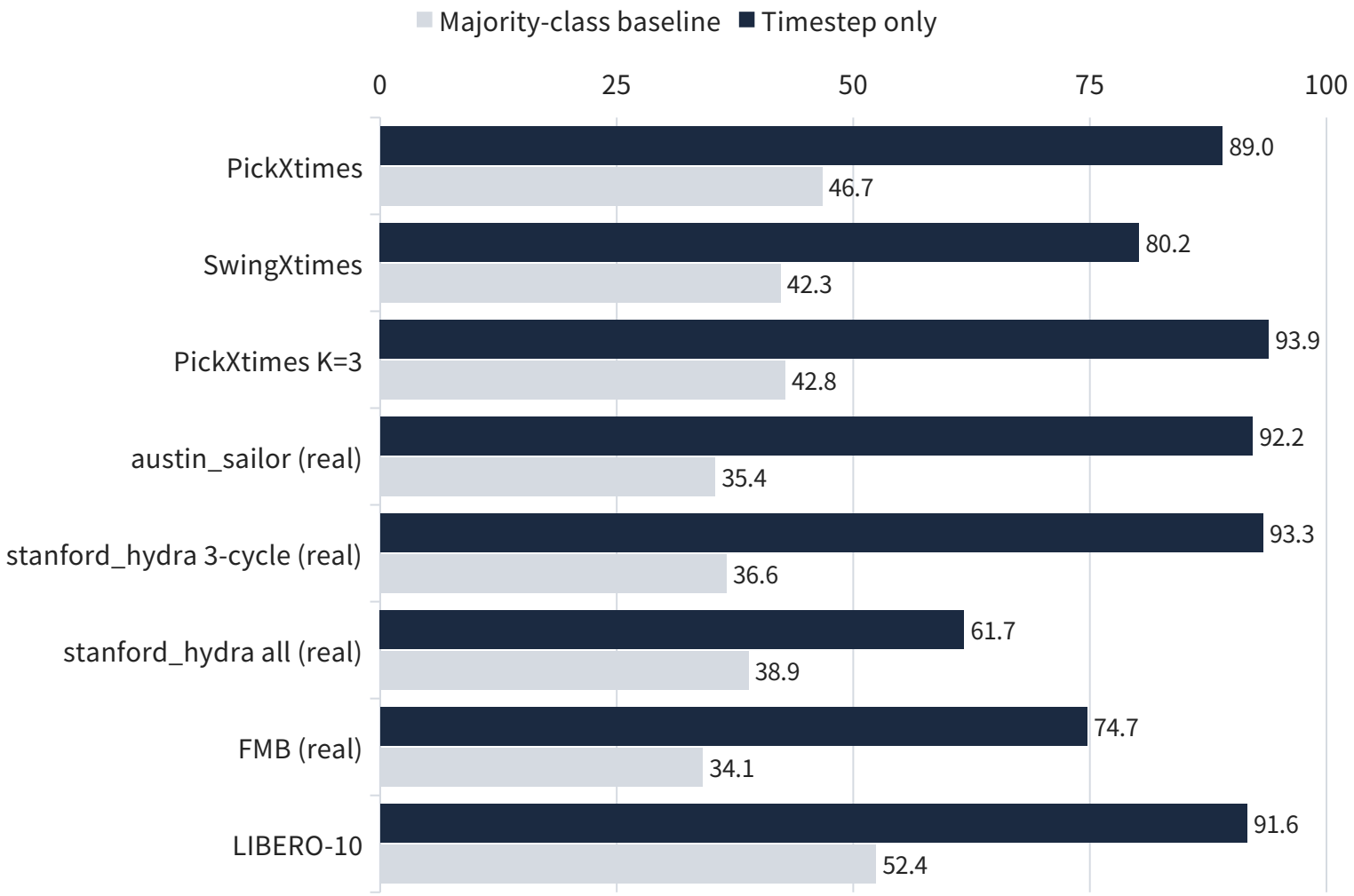
Recovers to 29/34 when hidden

Interpretation

Not random failure from “disturbance”, but systematic overestimation of progress: pressing the button when it should continue.

Mechanism ①

Steady demo pace: timestep predicts progress, so IL “reads the clock”



PickXtimes accuracy

89%

Baseline 46.7%

Real teleop 3-cycle

>92%

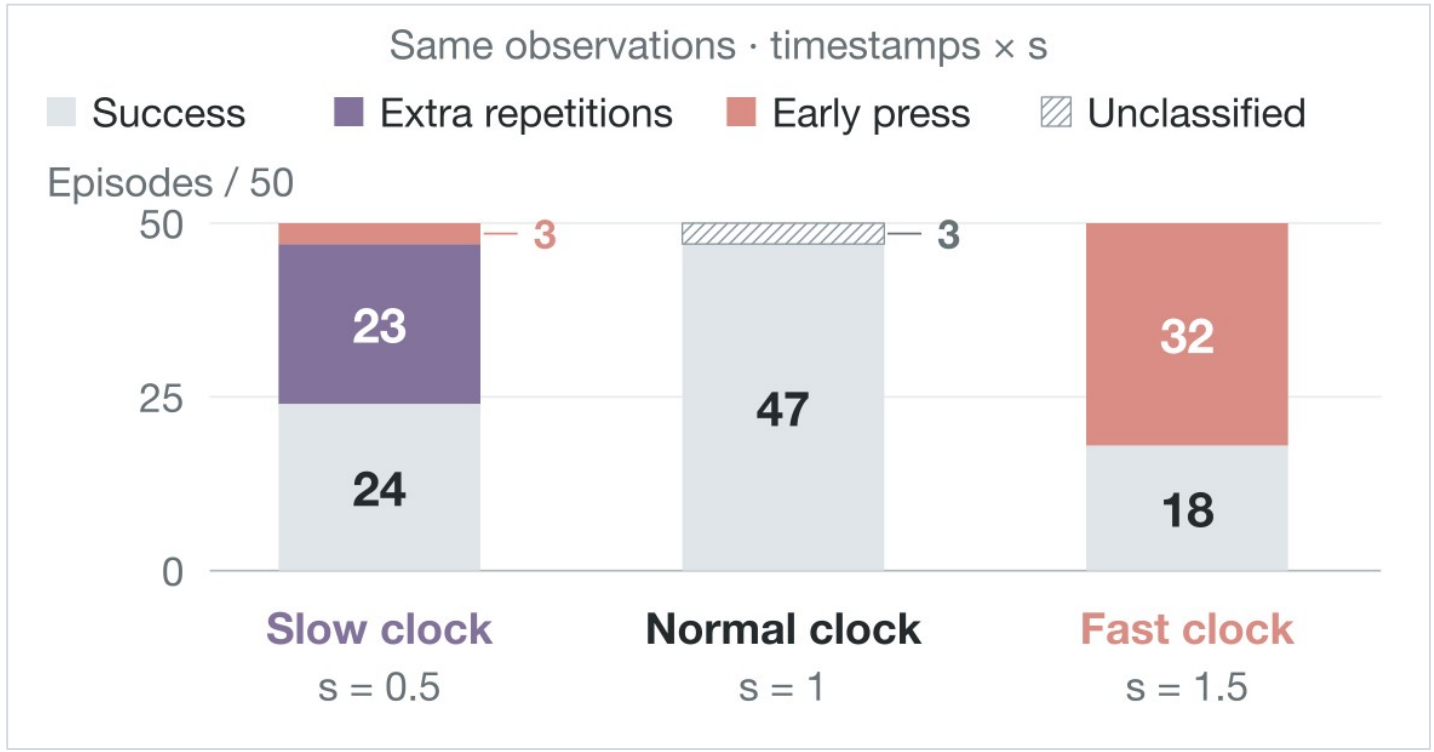
austin_sailor / stanford_hydra

No counterexamples to learn

Only 1 of 100 PickXtimes demos has a still segment (12 steps): training never penalizes “reading the clock”.

Mechanism ②

Scaling time alone steers failure: fast→early stop, slow→extra reps



s = 0.5 slow clock

24 successes

23 extra reps

s = 1.0 normal

47 successes

Reference

s = 1.5 fast clock

18 successes

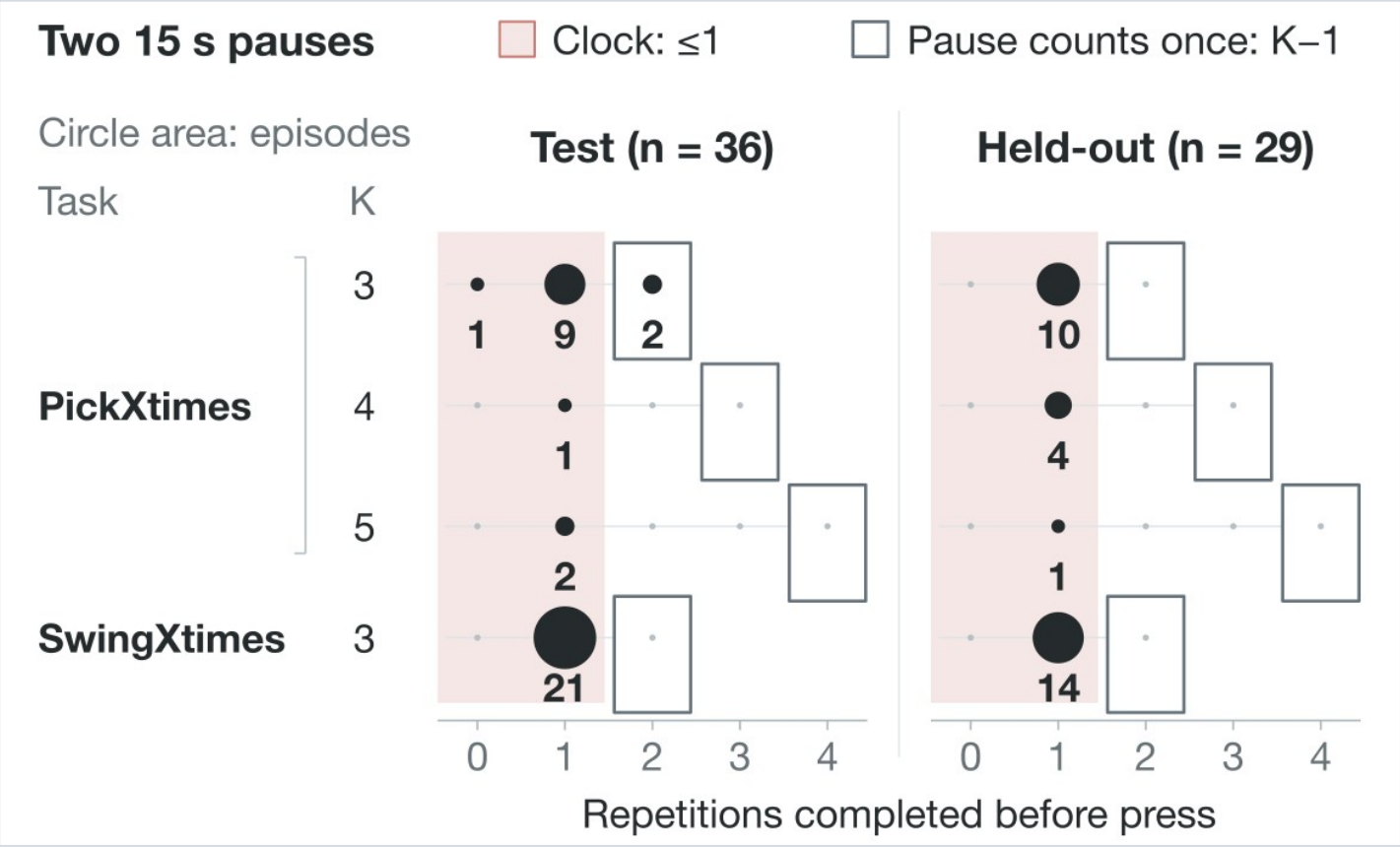
32 early presses

Null event = local clock speed-up

Time passes, content stays the same. The flip also appears with FrameSamp-Expert and a “frozen clock” (in-distribution layout).

Mechanism ③

The policy reads pause length, not merely that an event occurred



Two readings, two predictions

Reads clock: presses after ⩽1 rep

Pause = one rep: presses only after K−1 reps

Test set

34/36

Press after ⩽1 rep

Held-out set

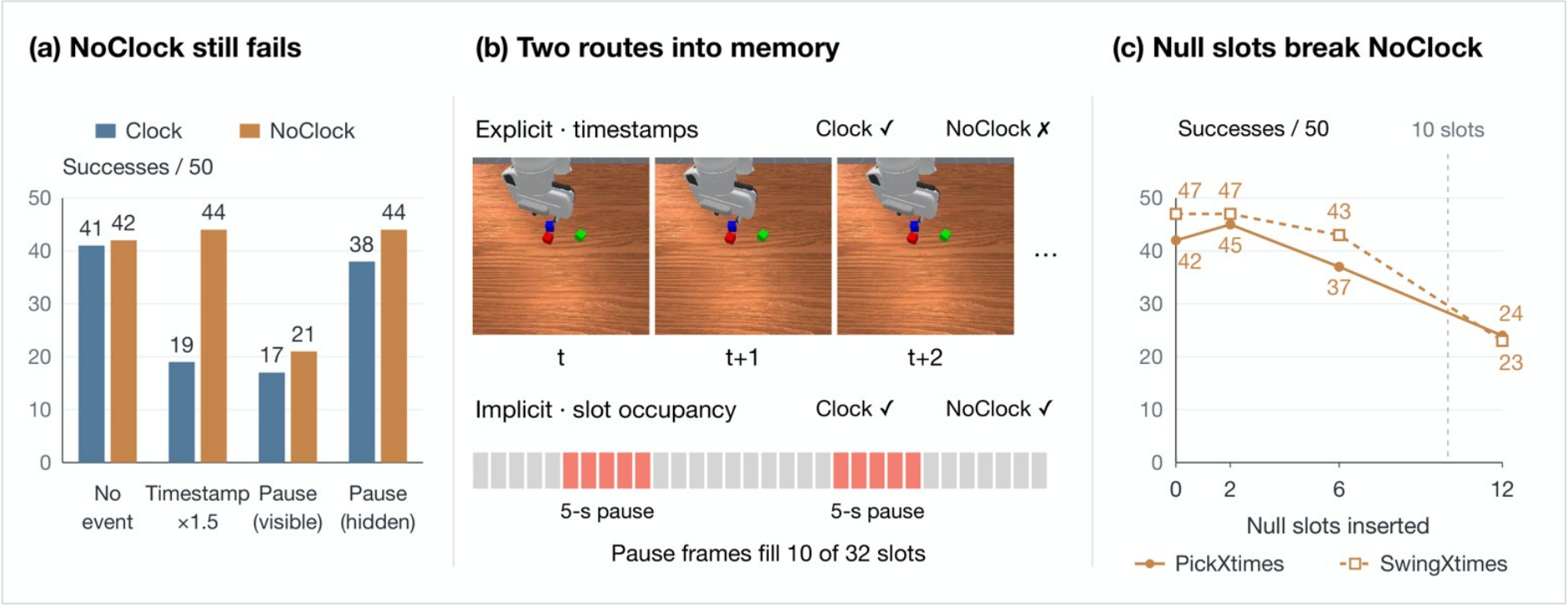
29/29

Criterion pre-set

Timestamps aren't the only path: storing frames with frozen timestamps also hurts, so frames carry part of the harm (Sec. E.1).

Mechanism ④

Dropping time encoding fails: null-event frames still occupy slots



NoClock ignores timestamps

Timestamps ×1.5: 44 / 50 (Clock: 19)

Yet pauses still break it

PickX 21 / 50, SwingX 14 / 50; hidden: 44 / 50

More empty slots, worse

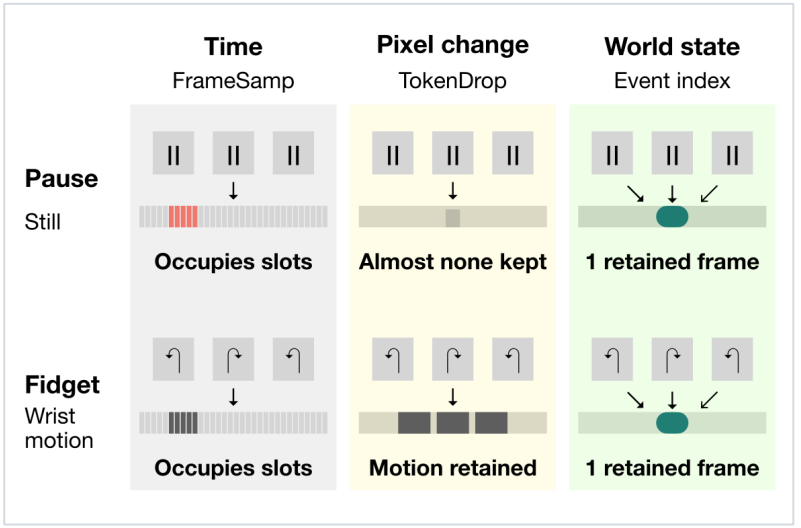
12 empty slots, no pause: PickX 42 → 24

Mechanism ⑤

Fragility depends on how memory is indexed, not whether it exists

Memory	Index	Pick pause	Pick fidget	Swing pause
FrameSamp-Modul	Time	47 → 16	47 → 16	44 → 12
FrameSamp-Expert	Time	44 → 14	44 → 17	49 → 11
FrameSamp-Context	Time	33 → 11	33 → 20	43 → 10
TokenDrop-Modul	Pixel change	39 → 30	39 → 21	45 → 27
MemER-style	Subgoal	38 → 39	38 → 38	24 → 23
Text sub-goals	Subgoal	46 → 48	46 → 47	–
TTT *	Recent	23 → 22	23 → 25	17 → 16
π0.5 no memory *	None	18 → 19	18 → 20	15 → 13
SimpleMemVLA †	Every step	50 → 47	50 → 38	48 → 36
LTM-π †	Every step	50 → 7	50 → 15	50 → 9

* Barely counts at all (K⩾2 nearly all fail), so being unaffected proves nothing. † Separate codebase, other backbones; 100-step events; SimpleMemVLA control is hidden.



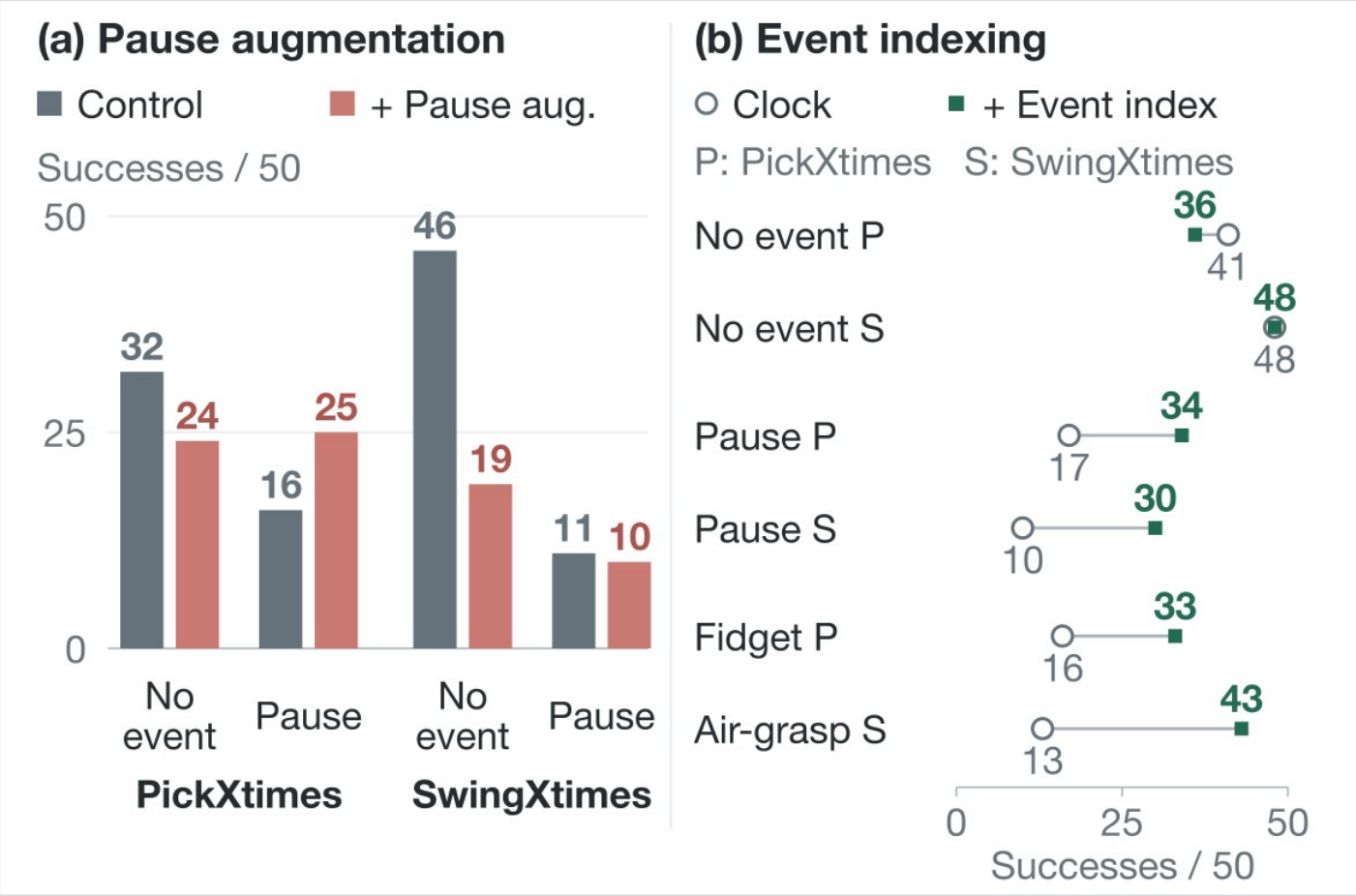
Each design fails where its index lets null events through

Time: pauses and fidgets both take slots

Pixel change: filters stillness, passes fidgets

Subgoal: immune, but weaker counting overall

Fix index, not data: event indexing recovers >½ of loss, no retraining



Data fix: pause augmentation fails

PickX pause 16 → 25, but no-event first drops 32 → 24

SwingX no-event 46 → 19; pause 11 → 10

Held-out K⩾2: 35/40 → 7/40, counting broken

Index fix: event indexing works

Pause: PickX 17 → 34, SwingX 10 → 30

Fidget: PickX 16 → 33

Air-grasp: SwingX 13 → 43

Cost: PickX no-event 41 → 36; replicated on held-out seeds and the released policy

Fix

Event indexing: advance only on change, blind spots

① Loop detection

Configuration returns to a kept frame, scene unchanged, no grasp/release or contact switch in between

② Compression

Collapse the segment into 1 frame; long stillness keeps only the first 5 frames

③ Re-timing

Renumber time positions over kept frames: memory advances only with events

Signals: robot mask (sim seg.), gripper width, net arm force; shared tolerances for all 16 tasks. A diagnostic, not a deployable detector.

Boundaries from the same criterion

No net cost on other 15 tasks: 316 vs 317 / 750

Hits real repetitions

In PickXtimes, returning to the original state after placing a cube is also a "loop": notable no-event cost for the released policy, plus induced extra reps (0/250 → 32/300)

Non-looping null events

Only about half of one-way air-grasps are removed; random micro-pauses (most deployment-like) barely recover

Duration is the task

In TimedTransfer / StopCube, compressing pauses is wrong by construction; such tasks still need a clock

Extensions

Video tasks hurt too; benchmarks may test clocks

RouteStick • two 15 s pauses

33 → 7

Hidden: 36; null frames evict video slots

Video pushed out of memory

Video and execution share 32 time-sampled slots (RouteStick: ~24 → ~10 slots)

Starvation control: no pause, video given equally few slots; harm reproduced (4)

Protection control: pause but keep video slots; recovers (32)

MIKASA-Robo (exploratory)

BlinkCount: light blinks K times, robot presses button K times

In demos, cue duration maps one-to-one to K, with non-overlapping ranges

Cue duration fixed at 50 steps: success 24 → 6; remaining successes concentrate at K=5, whose usual duration is closest to 50

Caveat: base success only 48%, below the preset bar; exploratory evidence only

3 cheap memory probes

Null events

visible vs hidden control

Clock scaling

Alter timestamps only; see failure direction

Fixed-duration cues

Stop duration leaking the answer

Assessment

Solid causal chain and stats; sim-only, fix more probe than solution

Strengths

- Concrete problem relevant to real deployment: waits, pauses and hesitation are common
- Complete causal chain: visible/hidden → clock scaling → NoClock → cross-design → fix
- Rigorous stats: paired McNemar, Holm correction, many pre-registered predictions, misses reported honestly
- Provides a transferable diagnostic protocol for any memory policy

Limitations

- All causal evidence is from simulation with scripted null events; real data shows only correlation
- Analysis centers on one family (MME-VLA); results for two independent policies are less clean
- Event indexing relies on sim signals, and its criterion mirrors how null events are built; largely ineffective on random micro-pauses
- Most from-scratch models use one seed; reruns differ by up to 12 / 50

Conclusion

Memory should count events, not ticks

It records time

Time-filled memory records both “how long” and “what happened”, and treats the former as progress

Two pathways

Timestamps carry duration; slot occupancy lets null events crowd memory. Removing the clock is not enough

The right place to fix

Change the index, not the data: event indexing recovers most of the loss without retraining

Open questions

How to reliably detect “world change” with real sensors; in duration-dependent tasks, time should be an input